Aim: Assignment.

1.Which feature was most important in the decision?

ANS: Class attendance is the most important feature in the dataset, since it directly affects the decision. It has large mean and Standard deviation than the rest.

2.Try removing Internet_Hours, does the accuracy improve?

ANS: If we removed the Internet_Hours column, it will not affect the accuracy since it is a noise. Passed column is only directly related to Study_Hours column and RandomForest will ignore irrelevant data.

CODE:

```
X_new = df.drop(['Passed', 'Internet_Hours'], axis=1)
```

```
X_train_new, X_test_new, y_train_new, y_test_new = train_test_split(X_new, y, test_size=0.2, random_state=42)
```

```
model_new = RandomForestClassifier(random_state=42)
model_new.fit(X_train_new, y_train_new)
y_pred_new = model_new.predict(X_test_new)
print("Accuracy without Internet_Hours:", accuracy_score(y_test_new, y_pred_new))
print("Confusion Matrix:\n", confusion_matrix(y_test_new, y_pred_new))
print("Classification Report:\n", classification_report(y_test_new, y_pred_new))
```

OUTPUT:

```
Accuracy without Internet_Hours: 0.8
Confusion Matrix:
 [[12  2]
 [ 2  4]]
Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.86      0.86        14
           1       0.67      0.67      0.67         6

    accuracy                           0.80        20
   macro avg       0.76      0.76      0.76        20
weighted avg       0.80      0.80      0.80        20
```

3.Change number of trees (n_estimators), what happens?

ANS: n_estimators is used to tell RandomForest how many tress has to be made, where each tree made there own predictions.

If n_estimators around 50, predictions may be unstable, accuracy might drop.

If n_estimators 100+, accuracy becomes stable and reliable.

If n_estimators 500+, accuracy will hardly improve, but training takes longer.

CODE & OUTPUT:

```
model_test = RandomForestClassifier(n_estimators = 25, random_state = 42)
model_test.fit(X_train, y_train)

y_pred_test = model_test.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_test))
```

```
Accuracy: 0.8
```

```
model_test = RandomForestClassifier(n_estimators = 90, random_state = 42)
model_test.fit(X_train, y_train)

y_pred_test = model_test.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_test))
```

```
Accuracy: 0.8
```

```
model_test = RandomForestClassifier(n_estimators = 800, random_state = 42)
model_test.fit(X_train, y_train)

y_pred_test = model_test.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_test))
```

```
Accuracy: 0.8
```

4.Try max_depth or min_samples__split parameters.

CODE:

```
np.random.seed(10)
data = {
    'Study_Hours': np.random.normal(5, 2, 100).round(1),
    'Sleep_Hours': np.random.normal(6, 1.5, 100).round(1),
    'Internet_Hours': np.random.normal(3, 1.3, 100).round(1),
    'Class_Attendance': np.random.normal(50, 100, 100).round(1)
}

df = pd.DataFrame(data)
df['Passed'] = np.where((df['Study_Hours']*10 + df['Class_Attendance']) > 105, 1, 0)
noise_idx = np.random.choice(df.index, size=10, replace=False)
df.loc[noise_idx, 'Passed'] = 1 - df.loc[noise_idx, 'Passed']

X = df.drop('Passed', axis=1)
y = df['Passed']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

params = [
    {"max_depth": None, "min_samples_split": 2},
    {"max_depth": 3, "min_samples_split": 2},
    {"max_depth": 5, "min_samples_split": 5},
    {"max_depth": 10, "min_samples_split": 10}
]

for p in params:
    model = RandomForestClassifier(random_state=42, **p)
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    print(f"max_depth={p['max_depth']}, min_samples_split={p['min_samples_split']}, Accuracy={acc:.4f}")
```

OUTPUT:

```
max_depth=None, min_samples_split=2, Accuracy=0.8000
max_depth=3, min_samples_split=2, Accuracy=0.8000
max_depth=5, min_samples_split=5, Accuracy=0.8000
max_depth=10, min_samples_split=10, Accuracy=0.8000
```

5.Introduce a dummy irrelevant columns like “Favorite_Color” and see if it gets any importance.

CODE:

```
np.random.seed(10)
data = {
    'Study_Hours': np.random.normal(5, 2, 100).round(1),
    'Sleep_Hours': np.random.normal(6, 1.5, 100).round(1),
    'Internet_Hours': np.random.normal(3, 1.3, 100).round(1),
    'Class_Attendance': np.random.normal(50, 100, 100).round(1)
}
df = pd.DataFrame(data)

df['Passed'] = np.where((df['Study_Hours'] * 10 + df['Class_Attendance']) > 105, 1, 0)

noise_idx = np.random.choice(df.index, size=10, replace=False)
df.loc[noise_idx, 'Passed'] = 1 - df.loc[noise_idx, 'Passed']

df['Favorite_Color'] = np.random.randint(0, 5, size=len(df))  # Random integers 0-4

X = df.drop('Passed', axis=1)
y = df['Passed']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

importances = model.feature_importances_
features = X.columns
plt.figure(figsize=(8,5))
sns.barplot(x=importances, y=features)
plt.title("Feature Importance (with irrelevant feature)")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.show()
```

OUTPUT:

```
Accuracy: 0.8
```